

第一个 Shell 脚本

本章学习目标

- 掌握 Shell 脚本文件的创建与命名规范
- 理解 Shebang（#!）的作用与正确写法
- 掌握脚本的三种执行方式及其区别
- 学会使用基本的调试方法排查脚本错误
- 掌握注释的写法与编写规范

2.1 什么是 Shell 脚本？

2.1.1 从手动到自动

想象一下这个场景：

你每天早上到公司都要做同样的事情——开电脑、泡咖啡、打开邮箱、查看日程。如果有机器人能帮你自动完成这一切，你只需要按一个按钮，是不是很方便？

Shell 脚本就是这样的"自动化按钮"。它把你需要手动输入的多条命令写在一个文件里，执行这个文件就能自动按顺序完成所有操作。

2.1.2 脚本 vs 交互式命令

对比项	交互式命令	Shell 脚本
执行方式	手动逐条输入	自动批量执行
可重复性	每次都要重新输入	写一次，用无数次
适用场景	临时性、一次性任务	重复性、定时任务
出错处理	即时发现，即时修改	需要调试排查
类比	现场口头指挥	书面操作手册

2.1.3 Shell 脚本的本质

Shell 脚本本质上就是一个纯文本文件，里面按顺序写着一条条 Shell 命令。当你执行这个脚本时，Shell 解释器会逐行读取并执行这些命令。

就像厨师看菜谱做菜一样：菜谱（脚本）上写着"第一步切菜，第二步热油，第三步下锅翻炒"，厨师（Shell）就按照这个顺序一步步操作。

2.2 创建第一个脚本

2.2.1 脚本文件的命名规范

在创建脚本之前，我们需要了解命名规范：

基本规则：

规则	正确示例	错误示例	说明
使用 <code>.sh</code> 后缀	<code>backup.sh</code>	<code>backup</code>	后缀便于识别文件类型
使用小写字母	<code>check_disk.sh</code>	<code>Check_Disk.sh</code>	Linux 区分大小写，小写更规范
单词间用下划线	<code>system_monitor.sh</code>	<code>system-monitor.sh</code>	下划线比连字符更常用
避免特殊字符	<code>log_clean.sh</code>	<code>log clean.sh</code>	空格和特殊字符会引起问题
名称要有意义	<code>backup_mysql.sh</code>	<code>script1.sh</code>	让人一看就知道脚本用途

常见命名模式：

动作_对象.sh	例如: backup_database.sh (备份数据库)
对象_动作.sh	例如: nginx_restart.sh (重启nginx)
check_xxx.sh	例如: check_disk.sh (检查磁盘)
deploy_xxx.sh	例如: deploy_app.sh (部署应用)

2.2.2 创建脚本文件

现在让我们动手创建第一个脚本！

【实操练习 2-1】创建第一个 Shell 脚本

```
# 1. 创建脚本存放目录（养成好习惯，脚本集中存放）
mkdir -p ~/scripts
cd ~/scripts

# 2. 使用 vim 创建脚本文件
vim hello.sh
```

在 vim 中输入以下内容：

```
#!/bin/bash
# 文件名: hello.sh
# 功能: 我的第一个Shell脚本
# 作者: 你的名字
# 日期: 2024-01-01

echo "Hello, Shell!"
echo "这是我的第一个脚本"
echo "当前时间是: $(date)"
echo "当前用户是: $(whoami)"
echo "当前目录是: $(pwd)"
```

保存并退出 vim（按 `Esc`，输入 `:wq`，按回车）。

脚本内容解析：

<code>#!/bin/bash</code>	← Shebang, 指定解释器 (下一节详解)
<code># 这是注释</code>	← 以 <code>#</code> 开头的是注释, 不会被执行
<code>echo "Hello, Shell!"</code>	← <code>echo</code> 命令, 输出文字到屏幕
<code>\$(date)</code>	← 命令替换, 执行 <code>date</code> 命令并插入结果

2.3 Shebang 详解

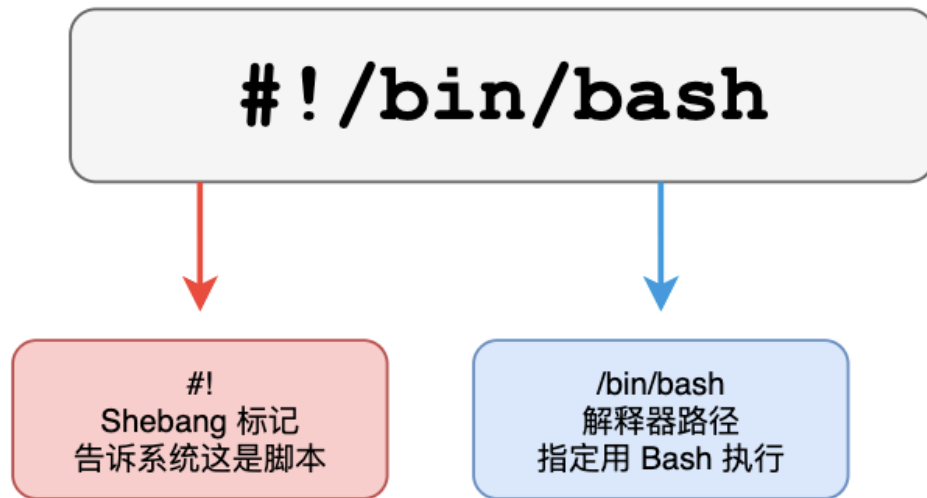
2.3.1 什么是 Shebang?

脚本第一行的 `#!/bin/bash` 叫做 **Shebang** (也叫 Hashbang), 它告诉系统用哪个解释器来执行这个脚本。

名称由来:

- `#` 在英文中叫 "hash" 或 "sharp"
- `!` 在英文中叫 "bang"
- 合起来就是 "Hash-Bang", 简称 "Shebang"

Shebang 结构解析



常见 Shebang 写法:

```
#!/bin/bash
```

← Bash (最常用)

```
#!/usr/bin/env bash
```

```
#!/bin/sh
```

← POSIX Shell (兼容性好) ← 推荐写法 (自动查找)

图2-1: Shebang 结构解析

2.3.2 常见的 Shebang 写法

Shebang	说明	适用场景
<code>#!/bin/bash</code>	使用 /bin/bash 执行	最常用, 适合大多数 Linux 系统
<code>#!/bin/sh</code>	使用 /bin/sh 执行	需要 POSIX 兼容性时使用
<code>#!/usr/bin/env bash</code>	自动查找 bash 位置	跨平台脚本 (推荐)
<code>#!/usr/bin/env python3</code>	使用 Python3 执行	Python 脚本

2.3.3 为什么推荐 `#!/usr/bin/env bash`?

不同系统中 Bash 的安装位置可能不同：

- CentOS/RHEL： `/bin/bash`
- 某些 Unix 系统： `/usr/local/bin/bash`
- macOS (Homebrew)： `/opt/homebrew/bin/bash`

使用 `#!/usr/bin/env bash` 可以让系统自动在 `PATH` 环境变量中查找 bash 的位置，提高脚本的可移植性。

【实操练习 2-2】查看 Bash 的位置

```
# 查看 bash 的实际位置
which bash

# 查看 env 命令的位置
which env

# 使用 type 命令查看
type bash
```

预期输出：

```
[root@localhost ~]# which bash
/usr/bin/bash

[root@localhost ~]# which env
/usr/bin/env
```

2.3.4 Shebang 的注意事项

 重要提醒：

1. Shebang 必须在脚本的**第一行**
2. `#!` 之间**不能有空格**
3. 路径必须是**绝对路径**
4. Shebang 行的行尾**不能有多余空格**

错误示例：

```
# 错误1：不在第一行
# 这是注释
#!/bin/bash           # 错！Shebang 之前有其他行

# 错误2：# 和 ! 之间有空格
# !/bin/bash          # 错！这样系统不识别

# 错误3：使用相对路径
#!bash                # 错！必须用绝对路径
```

2.4 脚本的执行方式

写好脚本后，我们需要执行它。有三种主要的执行方式，各有特点。

2.4.1 三种执行方式概览

Shell 脚本的三种执行方式

方式一：直接执行

```
chmod +x script.sh
./script.sh
```

- ✓ 需要先添加执行权限
- ✓ 启动新的子进程执行
- ✓ 依赖 Shebang 指定解释器
- ★ 最常用的方式

方式二：解释器执行

```
bash script.sh
sh script.sh
```

- ✓ 不需要执行权限
- ✓ 启动新的子进程执行
- ✓ 忽略 Shebang (手动指定)
- ★ 调试测试时常用

方式三：当前Shell执行

```
source script.sh
. script.sh
```

- ✓ 不需要执行权限
- ⚠ 在当前Shell中执行
- ⚠ 变量会影响当前环境
- ★ 加载配置文件时常用

⚡ 关键区别：子进程 vs 当前进程

方式一、二：子进程执行

脚本在新的子进程中运行，脚本中定义的变量、cd 切换的目录等都不会影响当前 Shell 环境。脚本结束后，子进程销毁。

方式三：当前进程执行

脚本在当前 Shell 中直接运行，脚本中定义的变量、cd 切换的目录等都会保留在当前环境中。常用于加载环境变量配置。

图2-2：脚本的三种执行方式对比

2.4.2 方式一：添加执行权限后直接执行

这是最标准、最常用的执行方式。

【实操练习 2-3】使用执行权限运行脚本

```
# 进入脚本目录
cd ~/scripts

# 查看脚本当前权限
ls -l hello.sh

# 添加执行权限
chmod +x hello.sh

# 再次查看权限（注意 x 的变化）
ls -l hello.sh

# 执行脚本
./hello.sh
```

执行过程解析：


```
[root@localhost scripts]# ls -l hello.sh
-rw-r--r--. 1 root root 198 Jan  1 10:00 hello.sh      # 没有 x 权限

[root@localhost scripts]# chmod +x hello.sh

[root@localhost scripts]# ls -l hello.sh
-rwxr-xr-x. 1 root root 198 Jan  1 10:00 hello.sh      # 有了 x 权限

[root@localhost scripts]# ./hello.sh
Hello, Shell!
这是我的第一个脚本
当前时间是: Mon Jan  1 10:05:00 CST 2024
当前用户是: root
当前目录是: /root/scripts
```

关于 `./` 的说明:

为什么要加 `./` 呢? 因为 Linux 系统出于安全考虑, 默认不在当前目录查找可执行文件。`./` 明确告诉系统"就在当前目录找这个脚本"。

```
# 以下两种写法等效
./hello.sh                # 相对路径
/root/scripts/hello.sh    # 绝对路径
```

2.4.3 方式二：使用解释器直接执行

这种方式不需要给脚本添加执行权限, 常用于临时测试或调试。

【实操练习 2-4】使用 bash 命令执行脚本

```
# 创建一个没有执行权限的新脚本
cat > test_exec.sh << 'EOF'
#!/bin/bash
echo "测试脚本执行"
echo "当前进程ID: $$"
EOF

# 查看权限 (没有 x)
ls -l test_exec.sh
```

```
# 尝试直接执行（会失败）
./test_exec.sh

# 使用 bash 命令执行（成功）
bash test_exec.sh

# 也可以使用 sh 执行
sh test_exec.sh
```

执行结果：

```
[root@localhost scripts]# ls -l test_exec.sh
-rw-r--r--. 1 root root 73 Jan  1 10:10 test_exec.sh

[root@localhost scripts]# ./test_exec.sh
-bash: ./test_exec.sh: Permission denied      # 权限拒绝

[root@localhost scripts]# bash test_exec.sh
测试脚本执行
当前进程ID: 12345                                # 成功执行
```

2.4.4 方式三：source 或 . 命令执行

这种方式在当前 Shell 环境中执行脚本，脚本中的变量和操作会影响当前环境。

【实操练习 2-5】理解 source 执行的特点

```
# 创建一个设置变量和切换目录的脚本
cat > env_test.sh << 'EOF'
#!/bin/bash
# 定义一个变量
MY_VAR="Hello from script"
echo "脚本内 MY_VAR = $MY_VAR"

# 切换目录
cd /tmp
echo "脚本内当前目录: $(pwd)"
EOF
```

```

# 记录当前目录和变量状态
echo "执行前 MY_VAR = $MY_VAR"
echo "执行前目录: $(pwd)"
echo "---使用 bash 执行---"

# 使用 bash 执行（子进程方式）
bash env_test.sh

# 检查当前环境
echo "bash执行后 MY_VAR = $MY_VAR"
echo "bash执行后目录: $(pwd)"
echo "---使用 source 执行---"

# 使用 source 执行（当前进程方式）
source env_test.sh

# 再次检查当前环境
echo "source执行后 MY_VAR = $MY_VAR"
echo "source执行后目录: $(pwd)"

```

执行结果分析：

执行前 MY_VAR =	# 变量为空
执行前目录: /root/scripts	
---使用 bash 执行---	
脚本内 MY_VAR = Hello from script	
脚本内当前目录: /tmp	
bash执行后 MY_VAR =	# 变量仍为空!
bash执行后目录: /root/scripts	# 目录没变!
---使用 source 执行---	
脚本内 MY_VAR = Hello from script	
脚本内当前目录: /tmp	
source执行后 MY_VAR = Hello from script	# 变量被设置了!
source执行后目录: /tmp	# 目录变了!

💡 重要结论：

- `bash script.sh`：脚本在子进程执行，不影响当前环境
- `source script.sh`：脚本在当前进程执行，会影响当前环境

source 的典型应用场景：

```
# 修改 .bashrc 后使其立即生效
source ~/.bashrc

# 也可以用点号 (.)，效果相同
. ~/.bashrc

# 加载自定义的环境变量配置
source /etc/profile.d/custom_env.sh
```

2.4.5 三种执行方式对比总结

执行方式	是否需要执行权限	执行环境	常用场景
<code>./script.sh</code>	✅ 需要	子进程	正式运行脚本
<code>bash script.sh</code>	❌ 不需要	子进程	测试和调试脚本
<code>source script.sh</code>	❌ 不需要	当前进程	加载环境变量配置

2.5 脚本调试基础

程序不可能一次写对，调试是开发过程中必不可少的环节。本节介绍几种基本的调试方法。

2.5.1 使用 `bash -x` 调试

`-x` 选项会在执行每条命令前，先打印出该命令（带 `+` 前缀），方便追踪执行过程。

【实操练习 2-6】使用 bash -x 调试脚本

```
# 创建一个简单的调试示例脚本
cat > debug_demo.sh << 'EOF'
#!/bin/bash
name="Shell"
echo "Hello, $name"
current_date=$(date +%Y-%m-%d)
echo "Today is $current_date"
EOF

# 正常执行
echo "=== 正常执行 ==="
bash debug_demo.sh

# 调试模式执行
echo "=== 调试模式 ==="
bash -x debug_demo.sh
```

输出对比：

```
=== 正常执行 ===
Hello, Shell
Today is 2024-01-01

=== 调试模式 ===
+ name=Shell
+ echo 'Hello, Shell'
Hello, Shell
+ date +%Y-%m-%d
++ date +%Y-%m-%d
+ current_date=2024-01-01
+ echo 'Today is 2024-01-01'
Today is 2024-01-01
```

+ 号表示这是将要执行的命令

命令替换的内部命令也显示

++ 表示嵌套的命令替换

2.5.2 使用 bash -n 语法检查

-n 选项只检查语法错误，不实际执行脚本。

【实操练习 2-7】使用 `bash -n` 检查语法

```
# 创建一个有语法错误的脚本
cat > syntax_error.sh << 'EOF'
#!/bin/bash
echo "Start"
if [ $a -eq 1 ]
then
    echo "a is 1"
# 故意漏掉 fi
echo "End"
EOF

# 使用 -n 检查语法
bash -n syntax_error.sh
```

输出：

```
syntax_error.sh: line 8: syntax error: unexpected end of file
```

语法检查发现了缺少 `fi` 的错误。

2.5.3 在脚本内部开启调试

除了在命令行使用 `-x`，还可以在脚本内部控制调试的范围：

```
#!/bin/bash

echo "这部分不调试"

set -x          # 从这里开始调试
echo "调试区域开始"
name="test"
echo "name = $name"
set +x          # 从这里关闭调试

echo "这部分也不调试"
```

常用的 set 选项：

选项	作用	说明
set -x	开启调试	显示执行的每条命令
set +x	关闭调试	停止显示命令
set -e	遇错退出	命令返回非零状态时立即退出
set -u	变量未定义报错	使用未定义变量时报错而非空值

2.5.4 使用 echo 打印调试信息

最简单但很有效的调试方法就是在关键位置添加 echo 输出：

```
#!/bin/bash

echo "[DEBUG] 脚本开始执行"

name="Shell"
echo "[DEBUG] name 变量值为: $name"

result=$((10 / 2))
echo "[DEBUG] 计算结果为: $result"

echo "[DEBUG] 脚本执行完毕"
```

 **调试技巧：**使用 [DEBUG] 这样的前缀可以方便地用 grep 过滤调试信息，或者在最终版本中快速删除这些调试行。

Shell 脚本调试方法

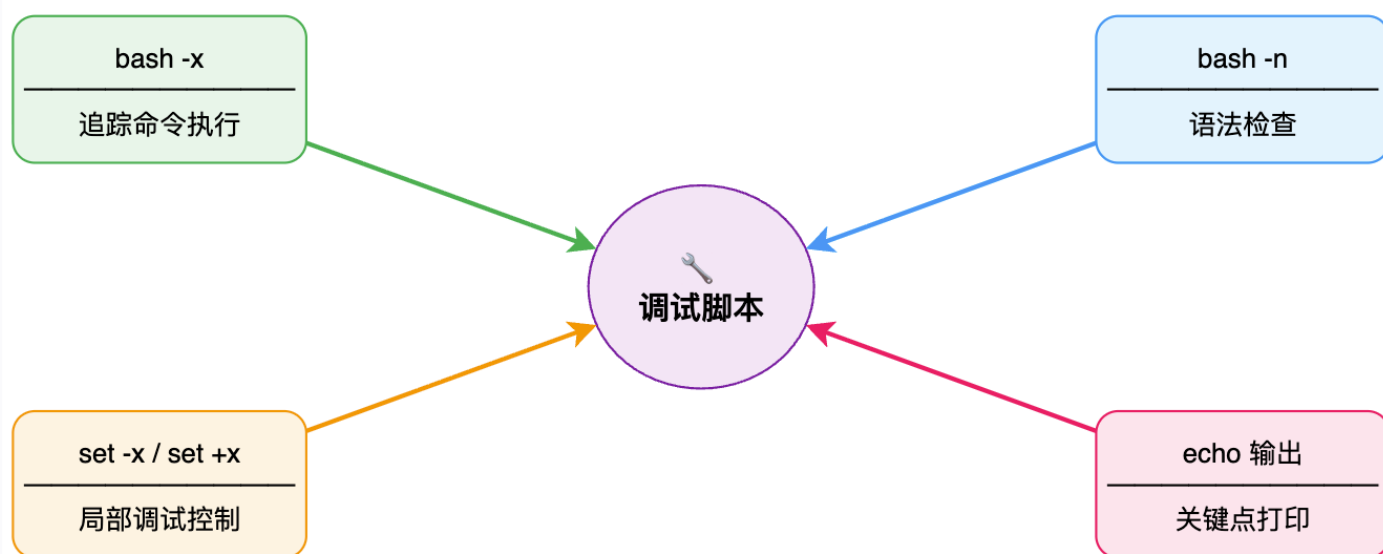


图2-3: 脚本调试方法概览

2.6 注释的写法与规范

好的注释能让代码更易于理解和维护，这是专业程序员的基本素养。

2.6.1 注释的基本语法

在 Shell 脚本中，以 `#` 开头的内容就是注释，不会被执行。

```
# 这是一行完整的注释
```

```
echo "Hello" # 这是行尾注释
```

```
# 可以用多行注释
```

```
# 解释一段
```

```
# 复杂的逻辑
```

⚠ 注意：Shell 没有像其他语言那样的多行注释语法（如 `/* */`），只能逐行使用 `#`。

2.6.2 文件头注释规范

每个脚本文件开头都应该有一个规范的头部注释，说明脚本的基本信息：

```
#!/bin/bash
#####
# 文件名: backup_mysql.sh
# 功能描述: 自动备份 MySQL 数据库
# 作者: 张三
# 创建日期: 2024-01-01
# 修改记录:
#   2024-01-05 张三 - 添加压缩功能
#   2024-01-10 李四 - 优化日志输出
# 使用方法: ./backup_mysql.sh [database_name]
# 依赖: mysqldump, gzip
#####
```

2.6.3 代码块注释

在复杂逻辑前添加注释，说明这段代码的作用：

```
#-----
# 1. 参数检查
#-----
if [ $# -lt 1 ]; then
    echo "Usage: $0 <database_name>"
    exit 1
fi

#-----
# 2. 执行备份
#-----
backup_file="/backup/mysql_$(date +%Y%m%d).sql"
mysqldump -u root -p "$1" > "$backup_file"

#-----
# 3. 压缩备份文件
#-----
gzip "$backup_file"
```

2.6.4 注释的最佳实践

✅ 好的做法	❌ 不好的做法
解释"为什么"这样做	只说明代码"做了什么"（代码本身已经说明）
说明复杂算法的思路	每一行都加注释
记录特殊情况和边界处理	写过时不更新的注释
标注 TODO 和待修复问题	写废话或自说自话

示例对比：

```
# ❌ 不好的注释：描述显而易见的事情
# 定义变量 name 为 Shell
name="Shell"

# 输出 Hello
echo "Hello"

# ✅ 好的注释：解释为什么
# 等待5秒确保服务完全启动后再检查状态
sleep 5

# 使用 -f 强制删除，避免交互式确认
rm -f /tmp/cache_*
```

2.7 综合实战：系统信息采集脚本

让我们综合运用本章学到的知识，编写一个实用的系统信息采集脚本。

【实操练习 2-8】编写系统信息采集脚本

```
# 创建脚本
vim ~/scripts/sysinfo.sh
```

输入以下内容：

```
#!/bin/bash
#####
# 文件名: sysinfo.sh
# 功能描述: 采集并显示系统基本信息
# 作者: 学员
# 创建日期: 2024-01-01
# 使用方法: ./sysinfo.sh
#####

#-----
# 定义分隔线函数（复用代码）
#-----
print_line() {
    echo "===== "
}

#-----
# 脚本主体开始
#-----
clear # 清屏

print_line
echo "      系统信息采集报告"
echo "      生成时间: $(date '+%Y-%m-%d %H:%M:%S')"
print_line

# 1. 主机信息
echo ""
echo " 【主机信息】 "
echo "   主机名: $(hostname)"
echo "   内核版本: $(uname -r)"
echo "   操作系统: $(cat /etc/redhat-release 2>/dev/null || cat /etc/os-release | grep PRETTY_NAME | cut -d'"'"' -f2)"

# 2. CPU信息
echo ""
echo " 【CPU信息】 "
echo "   CPU型号: $(grep 'model name' /proc/cpuinfo | head -1 | cut -d':' -f2 | sed 's/^[ \t]*//')"
echo "   CPU核数: $(grep -c 'processor' /proc/cpuinfo) 核"
```

```

# 3. 内存信息
echo ""
echo " 【内存信息】 "
mem_total=$(free -h | awk '/^Mem:/ {print $2}')
mem_used=$(free -h | awk '/^Mem:/ {print $3}')
mem_free=$(free -h | awk '/^Mem:/ {print $4}')
echo " 总内存: $mem_total"
echo " 已使用: $mem_used"
echo " 空闲: $mem_free"

# 4. 磁盘信息
echo ""
echo " 【磁盘使用情况】 "
df -h | grep -E '^/dev/' | awk '{printf " %-15s 总容量:%-8s 已用:%-8s
可用:%-8s 使用率:%s\n", $1, $2, $3, $4, $5}'

# 5. 网络信息
echo ""
echo " 【网络信息】 "
# 获取第一个非lo网卡的IP
ip_addr=$(ip addr show | grep -E 'inet ' | grep -v '127.0.0.1' | head
-1 | awk '{print $2}' | cut -d '/' -f1)
echo " IP地址: $ip_addr"

print_line
echo "采集完成! "
print_line

```

保存退出后执行：

```

# 添加执行权限
chmod +x ~/scripts/sysinfo.sh

# 执行脚本
~/scripts/sysinfo.sh

```

预期输出效果：

```
=====
```

系统信息采集报告

生成时间: 2024-01-01 10:30:00

【主机信息】

主机名: localhost

内核版本: 3.10.0-1160.el7.x86_64

操作系统: CentOS Linux 7 (Core)

【CPU信息】

CPU型号: Intel(R) Core(TM) i7-10700 CPU @ 2.90GHz

CPU核数: 2 核

【内存信息】

总内存: 1.8G

已使用: 512M

空闲: 1.0G

【磁盘使用情况】

/dev/sda1	总容量: 50G	已用: 8.2G	可用: 41G	使用率: 17%
-----------	----------	----------	---------	----------

【网络信息】

IP地址: 192.168.1.100

采集完成!

2.8 本章小结

本章我们学习了以下核心内容:

理论知识:

1. Shell 脚本是包含多条命令的文本文件, 用于自动化执行任务
2. Shebang (`#!`) 告诉系统用哪个解释器执行脚本, 必须放在第一行
3. 三种脚本执行方式: 直接执行 (需要权限)、解释器执行、source 执行
4. `bash script.sh` 在子进程执行, `source script.sh` 在当前进程执行

5. 良好的注释习惯让代码更易维护

实践技能：

1. 使用 `vim` 创建脚本文件
 2. 使用 `chmod +x` 添加执行权限
 3. 使用 `./`、`bash`、`source` 三种方式执行脚本
 4. 使用 `bash -x` 和 `bash -n` 调试脚本
 5. 编写规范的文件头注释和代码注释
-

2.9 课后练习

练习题 1（基础）：创建一个名为 `greet.sh` 的脚本，要求：

- 包含正确的 Shebang
- 输出 "早上好/下午好/晚上好"（根据当前时间）
- 输出当前登录用户名

练习题 2（基础）：解释以下两条命令的区别，并说明什么场景下使用哪种：

```
bash myscript.sh
source myscript.sh
```

练习题 3（进阶）：创建一个脚本 `env_demo.sh`，在脚本中定义变量 `TEST_VAR="hello"`。分别用 `bash` 和 `source` 两种方式执行，然后在终端中执行 `echo $TEST_VAR`，观察并解释结果的不同。

练习题 4（进阶）：编写一个脚本，故意包含一个语法错误（如缺少引号或 `fi`），然后：

1. 使用 `bash -n` 检查语法
2. 使用 `bash -x` 调试执行
3. 记录两种方式的输出差异

练习题 5（综合）：改进本章的 `sysinfo.sh` 脚本，添加以下功能：

- 显示系统运行时间（uptime）

- 显示当前登录用户数量
- 将输出结果同时保存到 `/tmp/sysinfo_日期.txt` 文件中